

SENTINELONE INTELLIGENCE LAYER

Implementation Plan and Overview

Sentry Agentic - Security Operations Centre

Management Report - 2026-07-31

1. The Problem

Before this work began, Sentry Agentic could not reliably answer questions about SentinelOne. When an analyst - or the system itself - needed to check something, such as how many threats existed or whether a device was online, there was no consistent, tested way of getting that answer. The system would work it out fresh each time, and its approach changed from one attempt to the next.

Real testing showed this clearly:

- None of the question types analysts actually ask had a proven, tested way to answer them correctly.
- There was no way to confirm a question had been answered using SentinelOne's real, live data, rather than the wrong source entirely.
- The same question could be handled differently from one conversation to the next.
- In one confirmed case, the system was asked how many threats existed and answered “zero,” when the real number was well above zero - because it had pulled the answer from the wrong internal system instead of SentinelOne, and stated it with full confidence.

This matters more as the project grows:

- If Sentry Agentic is eventually allowed to take action on its own - such as opening a support ticket or isolating an infected device - every one of those actions has to be backed by an answer that can genuinely be trusted.
- There was no way to test SentinelOne answers before relying on them in a live environment.
- A wrong “all clear” answer from a security tool is not a minor bug - it means a real threat could go unnoticed.
- Every new client environment would have needed its own manual trial and error, with no guarantee that accuracy learned once would carry over to the next.

Today's scope is also intentionally limited to a single, established client environment, with the system only able to answer questions, not take action. Supporting several client environments at once is a deliberate next step, not

something built yet.

2. Solution Implemented.

Two connected pieces of work were built to fix this, based on one rule: always check the real, live data - never guess, and never rely on memory.

Getting Accurate Answers From SentinelOne

- A direct, secure connection to SentinelOne's own official data interface, using one access key restricted to a single client environment - nothing broader.
- A clear, confirmed map of what actually exists in this environment - devices, alerts, vulnerabilities, and more - built only from what SentinelOne itself reports back, never assumed.
- A checklist of 13 common question types analysts ask, each paired with a proven, tested way to answer it correctly.
- A lightweight internal step that works out which question type a new question belongs to, without sending anything outside the system to do it.
- **Result:** what used to be guesswork is now a tested, repeatable process - all 13 question types now answer correctly against the live environment.

Checking the System's Own Answers

- Details like device names, security identifiers, and time periods are pulled out of a question using clear, predictable logic - never left to guesswork.
- Before answering, the system checks whether it genuinely has a proven way to answer this type of question. If it doesn't, it says so, rather than guessing.
- Every answer states exactly where the information came from, which environment it applies to, what time period it covers, and how many results were found.
- The system tells the difference between “nothing happened” and “we couldn't check that” - an empty result is only ever reported as clean once that's genuinely confirmed.
- **Result:** the one wrong “all clear” answer found during testing has not recurred - all 13 question types now check out correctly.

The next two pieces of work - teaching the system to carry out full investigations, and eventually letting it take action such as creating tickets - build directly on top of this foundation and have not started yet. They're covered later in this report.

3. Why This Approach, Rather Than a Simpler One

The faster option would have been to give the system open access to SentinelOne's tools and let it work out each question using its own judgment, every time. That is, in effect, what was in place before, and it's what produced the wrong "zero threats" answer above. It has four real problems:

- **It isn't consistent.** The same question can be handled differently each time, which makes it impossible to test or trust in advance.
- **It isn't reliably accurate.** Without a proven path for each question type, the system has no way to confirm it used the right source - the wrong-answer case is the expected outcome of this approach, not a rare exception.
- **It doesn't scale.** Every new client or data source would need its own manual tuning from scratch, with no guarantee that lessons learned would carry over.
- **It can't safely support automation.** The system cannot be allowed to open a ticket or take action based on a guess - there has to be a clear line between "this was proven" and "this was improvised."

The approach taken here trades a slower start for something that pays off repeatedly: once a question type is proven accurate, every future client, every future data source, and every future automated action can reuse that same guarantee, instead of rebuilding trust from nothing each time. Doing this careful work now - while the system can only answer questions and cannot yet act - is far cheaper and safer than trying to add the same guarantees to a system that is already acting on its own.

4. What This Project Is Trying to Achieve

This work has two overall goals. The first - giving Sentry Agentic a reliable, checkable understanding of SentinelOne - has two parts. The first part, making sure every question is answered correctly using real data, is **complete** and now live in the product. The second part, teaching the system to carry out full, multi-step investigations - such as fully triaging a threat or reconstructing an attack - is fully planned but **has not started**.

The second overall goal - letting Sentry Agentic take action on its own, such as opening a support ticket or isolating an infected device - **has not started** either, and the specific ticketing system it would use has not yet been decided.

Sentry Agentic already has a broader team of AI specialists built into the platform - for triage, investigation, threat hunting, and more - used across all of its integrations. This project's job isn't to build new specialists; it's to make sure the ones that touch SentinelOne are working from verified, accurate information.

5. What This Runs On

Sentry Agentic runs as a set of connected services, packaged so they behave the same way in development and in production. A database stores every finding, decision, and action the system takes, so nothing is lost and everything can be reviewed later. A fast in-memory cache speeds up repeated requests. A backend service handles all incoming questions and events. The interface analysts use is a web application.

Claude, Anthropic's AI model, is the platform's reasoning engine - but for SentinelOne questions specifically, it's now mostly a fallback: the tested, proven path answers most questions before Claude needs to reason about them at all.

SentinelOne itself is reached through its own official connector, running as a single process for this one client environment, with one restricted access key. Earlier plans assumed a more complex setup with a separate tunnel per client; that hasn't been needed yet, and remains an option for later, once several client environments are supported at once.

Three pieces were purpose-built for this project and didn't exist before: the internal question-matching step, the accuracy-checking layer, and the logic that automatically pulls details like device names and dates out of a question.

6. How a Question Gets Answered

When someone asks a SentinelOne question, here's what happens, in order:

1. The question comes in through chat.
2. An internal matching step checks whether it closely fits one of the 13 proven question types.
3. If it does, the system double-checks that there's a genuinely tested way to answer it - if not, it says so rather than guessing.
4. It pulls out any specific details it needs - a device name, a date range, and so on - directly from the question.
5. It fetches the real, current data from SentinelOne.
6. It formats the answer clearly, stating exactly where the information came from, what time period it covers, and how many results were found.
7. The answer goes straight back to the person who asked. The AI model itself only gets involved when a question doesn't match a proven type.

7. What Sentry Agentic Can Reliably Answer Today

There are 13 common categories of SentinelOne question analysts ask. **All 13 are now answered reliably**, using real, tested data:

- How many active threats or alerts exist right now, or over a chosen time period.
- Looking up a specific device by name or address.
- Reconstructing the full chain of related alerts tied to one incident.
- Checking which devices are offline, out of date, or otherwise unhealthy.
- Checking whether a specific known vulnerability affects any device in the environment.
- Counting the total number of devices being managed.
- Pulling the full detail on a specific alert.
- Listing current vulnerabilities by severity or status.

Five categories are not yet available, each for a specific, known reason - none of them are unexplained gaps:

- **Deep, free-form threat hunting** - blocked because a SentinelOne AI feature it depends on is currently unavailable on SentinelOne's own side, confirmed through direct testing, not something Sentry Agentic can fix.
- **Identity-related security questions** - this feature isn't licensed on this particular account, so there's genuinely nothing to report, not a gap in Sentry Agentic.
- **Questions about who changed a setting** - SentinelOne doesn't currently expose this kind of information through the connection Sentry Agentic uses.
- **Cloud misconfiguration questions** - checked, but this account doesn't appear to have any cloud security data populated yet.
- **Questions about Sentry Agentic's own internal case data** (rather than SentinelOne) - this is a control case confirming the system correctly knows when not to use SentinelOne, and it passes.

8. What Comes Next: Full Investigations

Once a question can be answered reliably, the next step is teaching Sentry Agentic to carry out full, multi-step investigations the way a human analyst would - always built strictly on top of the proven answers above, never by improvising a new way to fetch data. Six specialist roles and two supporting features are fully planned but not yet built:

- **Triage** - decide whether a threat is real, and how serious it is.

- **Investigator** - reconstruct the full attack chain and map it to known attack techniques.
- **Hunter** - proactively search for known attack patterns on a schedule.
- **Correlator** - spot patterns that connect separate-looking alerts into one bigger picture.
- **Threat Intelligence** - enrich a suspicious file or indicator with reputation and context.
- **Reporter** - assemble a full, audit-ready report.
- **Session export** - download any investigation as a PDF, Word document, or similar.
- **Daily briefing** - an automatic daily summary of what mattered in the environment over the last 24 hours.

None of these have started yet - they're specified in detail and ready to build once scheduled. See the Project Timeline for the planned build order.

9. How This Is Kept Secure

Security is addressed at every layer, from how passwords are stored up to how the AI itself is allowed to behave. Each item below names the actual safeguard in place, not just an intention.

Access Keys and Passwords

Every login, password, and access key - including the one used to reach SentinelOne - is stored in an encrypted vault, never left in a plain configuration file where it could be read directly.

Network Protection

Every response from the system includes standard web-security protections against common attacks, and traffic is protected against abuse and repeated requests.

Logins and Permissions

Production logins use secure, time-limited sessions with role-based permissions. There is a developer-only mode that skips all of this for local testing - it is never to be turned on in a live environment.

Approval Before Any Action

Every action the system could eventually take automatically - such as isolating a device or opening a ticket - already has a built-in approval system in place: each proposed action is logged with a confidence score and requires a person to approve it before anything happens, with a full record of who approved what and when. This exists today and isn't yet used by the SentinelOne connection, because that connection can currently only look things up, not act.

Automatic Security Checks

Every code change is automatically scanned for common security issues before it's accepted.

Specific to SentinelOne

- The access key used to reach SentinelOne is deliberately limited to one environment - it cannot see across multiple clients.
- The connection can currently only read information; it cannot isolate a device, change a setting, or take any action.
- Every entry on the question checklist names exactly which internal data source is off-limits for that type of question - specifically to prevent the kind of wrong-answer mistake described in Section 1.

Checks Built Into the AI Itself

A few of the more unusual, but important, safeguards:

- The system is built to refuse to answer confidently unless it genuinely has a tested way to do so - it cannot be talked into guessing.
- The step that pulls details like device names out of a question uses fixed, predictable logic rather than open-ended AI reasoning, which closes off a way a cleverly worded question could otherwise trick the system into an unintended request.
- The internal question-matching step runs entirely inside Sentry Agentic - question text is never sent externally just to work out what kind of question it is.
- Because a badly formed technical query can silently end the whole connection to SentinelOne, every query is limited to a safe time window and size, and a failed query is treated as a hard stop rather than something to guess and retry.

10. What's Deliberately Not Done Yet

- Deep threat hunting can't fully mature until a SentinelOne-side outage - confirmed, not on our end - is resolved.
- Identity security questions aren't available because that module isn't licensed on this account.
- Audit-log questions aren't available because SentinelOne doesn't expose that data through this connection.
- Cloud misconfiguration coverage is unconfirmed - checked once, found empty.
- Supporting more than one client environment at a time is deliberately deferred.
- Letting the system take action on its own hasn't started, and the ticketing system it will use hasn't been decided.

11. Related Documents

See **Sentry_Agentic_SentinelOne_Project_Timeline.pdf** for the complete, step-by-step build history, and **Sentry_Agentic_SentinelOne_Progress_Update.pdf** for current status, every issue found and fixed, and what's still open.

12. Conclusion

The first goal of this project - making sure Sentry Agentic gives accurate, checkable answers about SentinelOne - is done, and is being used in the live product today, answering every one of the 13 question types it was tested against. The next two goals - full investigations, and letting the system act on its own - are both fully planned but haven't started.

This order was chosen on purpose. Every safeguard described in this report exists to make it safe, later, to let the system recommend a ticket or an action with real confidence behind it. Building that foundation first, while the system can only answer questions and cannot yet act, is what makes the next two phases possible to do safely. What's needed to move forward is simply scheduling the next phase of work, and a decision on which ticketing system the automated-action phase should use - both covered in the Progress Update.